

Chapter 2: Regression

Austin R.J. Downey

Regression Approaches

In this chapter, we examine Linear Regression and two ways to estimate parameters:

- **Closed-form solution:** Solve directly for parameters by minimizing the cost function.
- **Gradient Descent:** Iteratively update parameters to reduce error.

Gradient Descent variants:

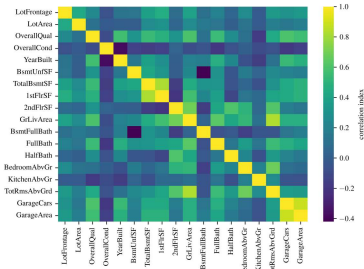
- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-batch Gradient Descent

Closed-form gives an exact solution immediately, while Gradient Descent converges through successive updates.

Data: Ames Housing Dataset Feature Correlations

Feature relationships:

- Shows correlations between inputs
- Helps identify important variables
- Useful for feature selection



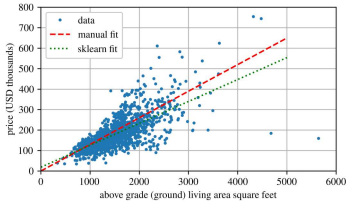
Correlation matrix of features.

A Simple Model

A clear trend is visible: larger homes tend to cost more.

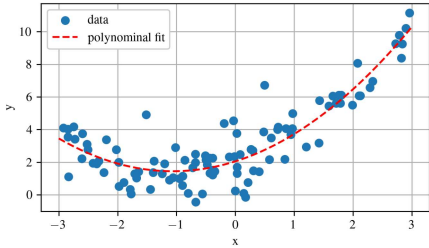
$$\text{price_model} = \theta_1 + \theta_2 \times \text{above_ground_living_area}$$

- θ_1 is the intercept
- θ_2 is the slope
- The model can represent any line



Ames housing data with trend lines.

Regression

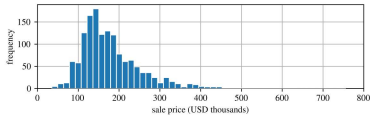


Regression models the relationship between an input variable x (data) and an output variable y (target). The goal is to learn a function that predicts y from x .

Data: Ames Housing Dataset

The Ames housing dataset contains:

- 2,930 home sales
- Data from 2006–2010
- Multiple feature types

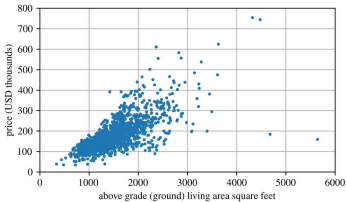


Histogram of sale prices.

Ames Housing Example

We want to predict house price using the Ames dataset.

- Input: above-ground living area
- Target: housing price
- Goal: find a linear relationship between them



Ames housing data.

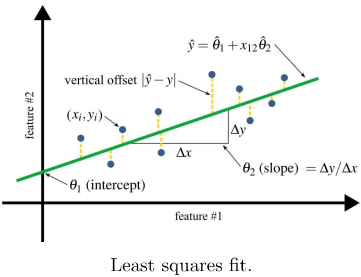
Notation

Notations

In terms of linear algebra notation; we use:

- $\mathbf{x}$: lowercase bold symbols denote vectors.
- $\mathbf{X}$: uppercase bold symbols denote matrices.
- In machine learning, we commonly use:
 - $\mathbf{X}$: the feature matrix (input data).
 - $\mathbf{y}$: the target vector (labels or outputs).
 - $\mathbf{x}^{(i)}$: the i^{th} instance in the dataset.
 - h : the hypothesis or prediction function, $\hat{\mathbf{y}} = h(\mathbf{X})$.
 - n : the number of features.
 - m : the number of training instances.
 - $\hat{x}$: an estimated value (“ x -hat”).

- Goal:
- Find parameters that minimize error
 - Fit a line closest to all data points
- Idea:
- Measure vertical distance to line
 - Square the errors
 - Sum over all points



Mean Squared Error (MSE)

A common cost function is Mean Squared Error:

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- m : number of data points
- Measures average squared prediction error
- Larger errors are penalized more heavily

For linear regression:

$$J = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

Matrix Formulation

We can write the linear model compactly:

$$\mathbf{y} = \mathbf{X}\theta + \epsilon$$

- $\mathbf{X}$: feature matrix
- θ : parameters
- $\mathbf{y}$: target values

Predictions:

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\theta}$$

- Goal:
- Find $\hat{\theta}$ that minimizes error

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$
$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Computational Complexity

- Normal Equation requires inverting $\mathbf{X}^T \mathbf{X}$ (an $n \times n$ matrix)
- Matrix inversion cost: $O(n^{2.4})$ to $O(n^3)$
- Doubling features $\Rightarrow \sim 5-8\times$ more computation

Warning

For very large feature sets (e.g., $n \sim 100,000$), the Normal Equation becomes extremely slow.

- Scales linearly with number of instances: $O(m)$
- Prediction is fast: linear in both m and n

Before tuning a model, we need a way to measure how well it fits the data.

- Define a **cost function**
- Penalizes differences between predictions and true values
- Lower cost = better fit

For linear regression:

- Use **Ordinary Least Squares (OLS)**
- Minimizes prediction error over all training data

Linear Regression Model

Consider a dataset with n observations and p features.

For each observation:

$$y_i = \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_p x_{ip} + \epsilon_i$$

- y_i : target value
- x_{ij} : feature values
- θ_j : model parameters
- ϵ_i : error term

The Normal Equation

Closed-form solution:

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

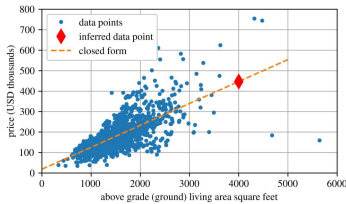
- Gives the exact best-fit parameters
- No iteration required
- Works well for small feature sets

For a simple model:

$$\hat{y} = \hat{\theta}_1 + x \hat{\theta}_2$$

This is just:

$$y = mx + b$$



Example 2.1: Ames Housing (Linear Regression)

- Fit a linear model to predict house price from living area
- Compare approaches:
 - Manual fit (visual intuition)
 - Closed-form (Normal Equation)
 - Gradient Descent
- In practice (scikit-learn):
 - `LinearRegression` (closed-form)
 - `SGDRegressor` (iterative)

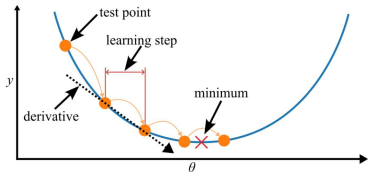
Next: methods that scale better for large feature sets or large datasets.

Gradient Descent

- Optimization algorithm to minimize a cost function
- Iteratively updates parameters

Idea:

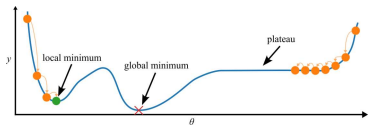
1. Compute the gradient (slope)
2. Move in direction of steepest descent
3. Repeat until minimum is reached



Gradient descent moving toward a minimum.

Challenges

- Cost functions may be complex:
 - Local minima
 - Plateaus
 - Ridges
- Starting point matters
- Convergence may be slow



Optimization challenges in complex landscapes.

For linear regression:

- MSE is convex
- One global minimum

Method Comparison

Different approaches to linear regression vary in speed, scalability, and flexibility.

algorithm	large m	in memory	large n	hyperparams	scaling	sklearn
normal equation	fast	yes	slow	0	no	<code>LinearRegression</code>
batch GD	slow	yes	fast	2	yes	n/a
stochastic GD	fast	no	fast	≥ 2	yes	<code>SGDRegressor</code>
mini-batch GD	fast	no	fast	≥ 2	yes	n/a

NOTE

After training, there is minimal difference between the models: these algorithms converge on similar solutions and make predictions in a nearly identical manner.

Gradient Computation

Compute partial derivatives:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^\top \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

Key Idea

Move in the opposite direction of the gradient to reduce error.

Vector form:

$$\nabla_{\theta} J(\theta) = \frac{2}{m} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y})$$

- Computes all slopes at once
- Direction of steepest ascent

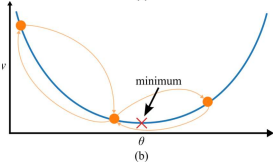
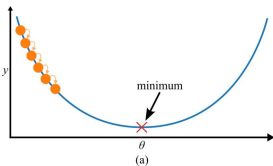
Warning

Uses the full dataset at every step $\rightarrow$ can be slow for large m .

Learning Rate

- Step size controlled by learning rate η
- Small $\eta \rightarrow$ slow convergence
- Large $\eta \rightarrow$ overshooting / divergence

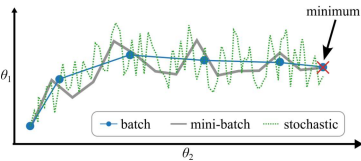
Choosing η is critical



Effect of different learning rates.

Comparing Gradient Descent Methods

- Different methods reach the minimum differently
- Trade-off between speed and stability
- Some methods are noisy but fast
- Others are stable but computationally expensive



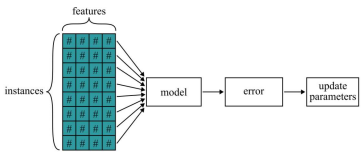
Paths taken by different methods.

All aim to minimize the same cost function

Batch Gradient Descent

- Uses the entire dataset at each step
- Minimizes the cost function over all data
- Based on Mean Squared Error (MSE)

$$J = \frac{1}{m} \sum_{i=1}^m (\theta^\top \mathbf{x}^{(i)} - y^{(i)})^2$$



Batch Gradient Descent flow.

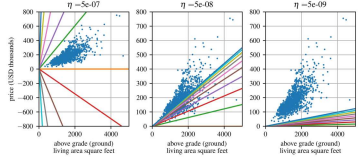
- Computes exact gradient each iteration
- Can be slow for large datasets

Parameter Update and Learning Rate

Update rule:

$$\theta^{(\text{next})} = \theta - \eta \nabla_{\theta} J(\theta)$$

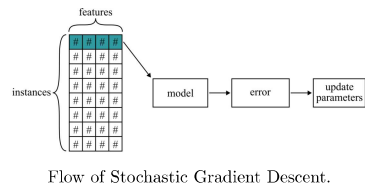
- η : learning rate (step size)
- Controls speed of convergence
- Too large $\rightarrow$ divergence
- Too small $\rightarrow$ slow convergence
- Just right $\rightarrow$ fast convergence



Effect of different learning rates.

Stochastic Gradient Descent

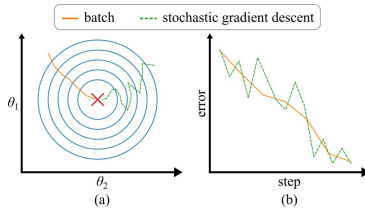
- Updates parameters using one sample at a time
- Uses $(x^{(i)}, y^{(i)})$ instead of full dataset
- Faster for large datasets
- Enables online learning



Flow of Stochastic Gradient Descent.

SGD Behavior

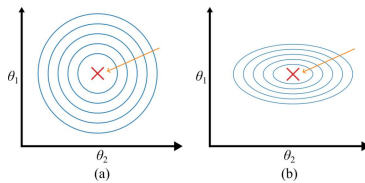
- Fast updates, but noisy path
- High variance in cost function
- Does not decrease smoothly
- Often oscillates around minimum
- Can still reach a good solution



Batch vs. stochastic gradient descent behavior.

Feature Scaling

- Different feature scales distort the cost surface
- Creates an elongated “valley”
- Equal scaling → fast convergence
- Unequal scaling → slow convergence



Effect of feature scaling on convergence.

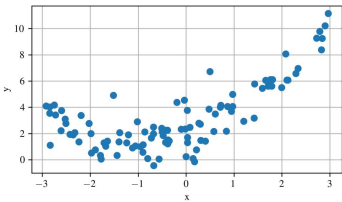
Gradient Descent becomes inefficient without scaling

Polynomial Regression

What if the data is not linear?
A quadratic model can represent curved behavior:

$$\hat{y} = ax^2 + bx + c$$

- Linear models underfit nonlinear data
- Polynomial terms allow curved fits
- Still linear in the parameters



Dataset generated from a quadratic relationship.

SGD Update Rule

Update rule:

$$\theta^{(\text{next})} = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$$

- Updates after every single example
- Avoids redundant computations
- More efficient than Batch GD for large m

Key Idea
Instead of waiting to see all data, update immediately using each new sample.

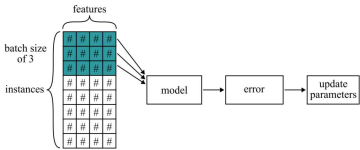
Mini-batch Gradient Descent

- Uses small random subsets (mini-batches)
- Between Batch GD and Stochastic GD

Update rule:

$$\theta^{(\text{next})} = \theta - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}, y^{(i:i+n)})$$

- Faster than Batch GD
- More stable than SGD
- Efficient on GPUs (matrix operations)



Flow of Mini-batch Gradient Descent.

Scaling Methods

Min-max scaling

$$\mathbf{X}' = \frac{\mathbf{X} - \mathbf{X}_{\min}}{\mathbf{X}_{\max} - \mathbf{X}_{\min}}$$

- Rescales to $[0, 1]$
- Sensitive to outliers

- Both improve Gradient Descent performance
- Standardization is most commonly used

Standardization

$$\mathbf{X}' = \frac{\mathbf{X} - \bar{\mathbf{X}}}{\sigma}$$

- Zero mean, unit variance
- Less sensitive to outliers

Tools
MinMaxScaler
StandardScaler

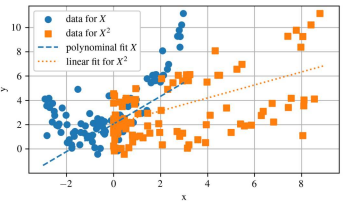
Feature Expansion

Polynomial regression works by expanding the feature space.

For example:

$$x \rightarrow [x, x^2]$$

- Add higher-order terms as new features
- Train a linear model on the expanded data
- Allows nonlinear behavior



Linear fits to the expanded feature set.

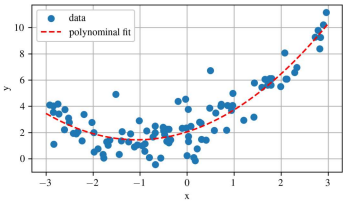
Polynomial Model Fit

The learned coefficients combine into:

$$\hat{y} = ax^2 + bx + c$$

- ▶ a : quadratic term
- ▶ b : linear term
- ▶ c : bias/intercept

The resulting model can approximate curved data.



Polynomial regression fit.

Combinatorial Explosion

Warning

Polynomial feature expansion can rapidly increase the number of features.

Number of expanded features:

$$\frac{(n + d)!}{d!n!}$$

- ▶ n : original number of features
- ▶ d : polynomial degree

Higher polynomial degrees can:

- ▶ Increase memory usage
- ▶ Slow training
- ▶ Cause overfitting

Overfitting

Training and testing on the same dataset is a major mistake.

- ▶ The model can memorize the training data
- ▶ Training accuracy may appear perfect
- ▶ Performance on new data may be poor

This problem is called:

Overfitting

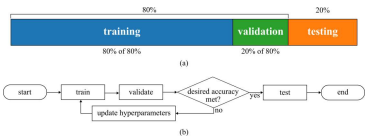
Goal:

- ▶ Build models that generalize to unseen data

Validation Sets

A validation set is often added to tune models.

- ▶ Training set:
 - ▶ Learn model parameters
- ▶ Validation set:
 - ▶ Tune hyperparameters
- ▶ Test set:
 - ▶ Final evaluation



Training, validation, and testing workflow.

Scikit-Learn:

```
train_test_split
```

Polynomial Features

Polynomial regression can also model feature interactions.

Example with two features a and b :

$[a, b]$

Degree-3 expansion:

$[a, b, a^2, b^2, a^3, b^3, ab, a^2b, ab^2]$

- ▶ Captures nonlinear relationships
- ▶ Captures interactions between variables
- ▶ Implemented with:

```
PolynomialFeatures(degree=d)
```

Example 2.2: Polynomial Regression

- ▶ Fit a nonlinear dataset using polynomial regression
- ▶ Add x^2 as an additional feature
- ▶ Train a linear model on the expanded feature set

Demonstrates how:

- ▶ Linear models can approximate curved relationships
- ▶ Polynomial terms improve model flexibility
- ▶ The resulting fit compares to the original data

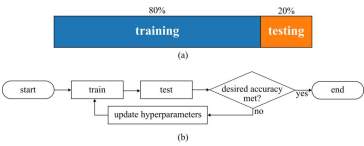
Tools used:

```
PolynomialFeatures      LinearRegression
```

Training and Testing Sets

Standard supervised learning workflow:

- ▶ Train model using training data
- ▶ Evaluate model using test data
- ▶ Test data must remain unseen during training



Notation:

$\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}$

Training/testing split.

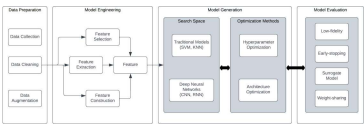
$\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}}$

Machine Learning Pipelines

Scikit-Learn pipelines combine multiple processing steps into a single workflow.

Typical steps:

- ▶ Data preprocessing
- ▶ Feature scaling
- ▶ Feature engineering
- ▶ Model training



Benefits:

- ▶ Cleaner code
- ▶ Consistent preprocessing
- ▶ Easier cross-validation
- ▶ Simplified hyperparameter tuning

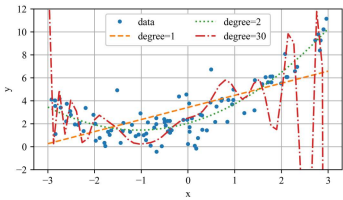
Example machine learning pipeline.

Learning Curves

Model complexity strongly affects performance.

- ▶ Low complexity:
 - ▶ Underfitting
- ▶ High complexity:
 - ▶ Overfitting
- ▶ Intermediate complexity:
 - ▶ Better generalization

Learning curves help diagnose model behavior.



Underfitting, good fit, and overfitting.

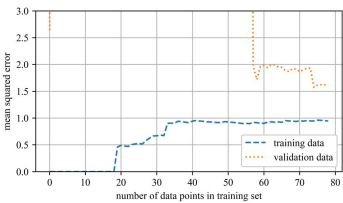
Overfitting

Characteristics of overfitting:

- ▶ Model is too complex
- ▶ Fits noise in the training data
- ▶ Excellent training performance
- ▶ Poor validation performance

Learning curves:

- ▶ Large gap between curves
- ▶ High variance



Learning curves for an overfitting model.

Interpreting Learning Curves

Learning curves compare:

Training Error vs. Validation Error

- ▶ Underfitting:
 - ▶ Both errors high
 - ▶ Curves close together
- ▶ Overfitting:
 - ▶ Training error low
 - ▶ Validation error much higher
- ▶ Good fit:
 - ▶ Both errors low
 - ▶ Curves converge

Regularized Linear Models

Overfitting can be reduced through:

Regularization

Idea:

- ▶ Constrain model complexity
- ▶ Prevent fitting noise in the data
- ▶ Improve generalization

For polynomial models:

- ▶ Use a lower polynomial degree

For linear models:

- ▶ Penalize large coefficient values

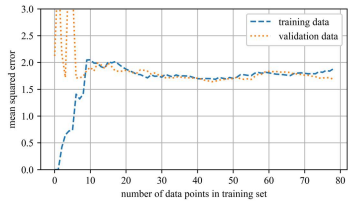
Underfitting

Characteristics of underfitting:

- ▶ Model is too simple
- ▶ Cannot capture data complexity
- ▶ Training and validation errors remain high

Learning curves:

- ▶ Curves are close together
- ▶ Errors plateau at high values



Learning curves for an underfitting model.

Well-Fit Model

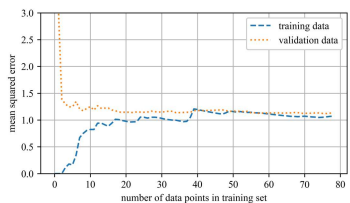
A good model balances:

- ▶ Bias
- ▶ Variance

Characteristics:

- ▶ Low training error
- ▶ Low validation error
- ▶ Small gap between curves

The model generalizes well to unseen data.



Learning curves for a well-fit model.

Example 2.3: Learning Curves

- ▶ Compare learning curves for:
 - ▶ Linear regression
 - ▶ Polynomial regression
- ▶ Plot training and validation errors as the training set grows
- ▶ Demonstrates:
 - ▶ Underfitting
 - ▶ Overfitting
 - ▶ Model generalization
 - ▶ Bias–variance tradeoff
- ▶ Shows how model complexity affects performance

Ridge Regression

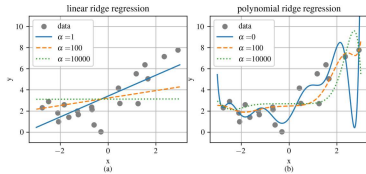
Ridge Regression adds a penalty term to the cost function:

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

- ▶ α : regularization strength
- ▶ Large $\alpha \rightarrow$ smaller weights
- ▶ Reduces overfitting

Note

The bias term θ_0 is not regularized.



Ridge regression with different α values.

Example 2.4: Ridge Regression

- ▶ Apply Ridge Regression to a high-degree polynomial model
- ▶ Add regularization to reduce overfitting
- ▶ Use a pipeline with:
 - ▶ `PolynomialFeatures`
 - ▶ `StandardScaler`
 - ▶ `Ridge`
- ▶ Demonstrates:
 - ▶ Smoother predictions
 - ▶ Reduced variance
 - ▶ Improved model stability
- ▶ Compare different values of α

Example 2.5: Early Stopping

- ▶ Train a high-degree polynomial model using:
 - ▶ Stochastic Gradient Descent
- ▶ Track:
 - ▶ Training error
 - ▶ Validation error
- ▶ Demonstrates:
 - ▶ How overfitting develops during training
 - ▶ How validation error identifies the optimal stopping point
 - ▶ Improved generalization through early stopping
- ▶ Training halts once validation performance begins degrading

Early Stopping

Early stopping is a form of regularization for iterative learning methods.

Idea:

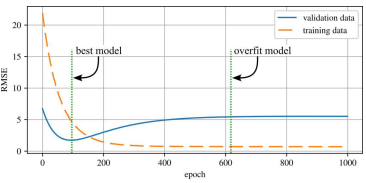
- ▶ Monitor validation error during training
- ▶ Stop training when validation error begins increasing

Benefits:

- ▶ Prevents overfitting
- ▶ Improves generalization
- ▶ Simple and computationally inexpensive

Key Idea

Stop training at the minimum validation error.



Early stopping prevents overfitting.